Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2025-26

Class:	**TE**	**Semester:**	**VI**
Course Code:	**CSL601**	**Course Name:**	**Data Analytics and Visualization Lab**

Name of Student:	**Shravani Sandeep Raut**
Roll No.:	**40**
Experiment No.:	**08**
Title of the Experiment:	**Data Visualization: Use Python libraries for visualization.**
Date of Performance:	**07/04/2026**
Date of Submission:	**10/04/2026**

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Meet Expect Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty: Ms. Odilia Gonsalves

Signature:

Date:

Experiment No. 8

Aim - Data Visualization: Use Matplotlib and seaborn Python libraries for visualization.

Objective: - To understand and apply the Matplotlib and seaborn python libraries for visualization using python.

Description-

Data visualization is an easier way of presenting the data, however complex it is, to analyze trends and relationships amongst variables with the help of pictorial representation.

The following are the advantages of Data Visualization

- Easier representation of compels data
- Highlights good and bad performing areas
- Explores relationship between data points
- Identifies data patterns even for larger data points

While building visualization, it is always a good practice to keep some below mentioned points in mind.

- Ensure appropriate usage of shapes, colors, and size while building visualization
- Plots/graphs using a co-ordinate system are more pronounced
- Knowledge of suitable plot with respect to the data types brings more clarity to the information
- Usage of labels, titles, legends and pointers passes seamless information the wider audience

Python Libraries

There are a lot of python libraries which could be used to build visualization like matplotlib, vispy, bokeh, seaborn, pygal, folium, plotly, cufflinks, and networkx. Of the many, matplotlib and seaborn seems to be very widely used for basic to intermediate level of visualizations.

Matplotlib

It is an amazing visualization library in Python for 2D plots of arrays, It is a multi-platform data visualization library built on NumPy arrays and designed to work with the broader SciPy stack. It was introduced by John Hunter in the year 2002. Let's try to understand some of the benefits and features of matplotlib

- It's fast, efficient as it is based on numpy and also easier to build
- Has undergone a lot of improvements from the open source community since inception and hence a better library having advanced features as well
- Well maintained visualization output with high quality graphics draws a lot of users to it
- Basic as well as advanced charts could be very easily built
- From the users/developers point of view, since it has a large community support, resolving issues and debugging becomes much easier

Seaborn

Conceptualized and built originally at the Stanford University, this library sits on top of matplotlib. In a sense, it has some flavors of matplotlib while from the visualization point, it is much better than matplotlib and has added features as well. Below are its advantages

- Built-in themes aid better visualization
- Statistical functions aiding better data insights
- Better aesthetics and built-in plots
- Helpful documentation with effective examples

Nature of Visualization

Depending on the number of variables used for plotting the visualization and the type of variables, there could be different types of charts which we could use to understand the relationship. Based on the count of variables, we could have

- Univariate plot(involves only one variable)
- Bivariate plot(more than one variable in required)

A Univariate plot could be for a continuous variable to understand the spread and distribution of the variable while for a discrete variable it could tell us the count

Similarly, a Bivariate plot for continuous variable could display essential statistic like correlation, for a continuous versus discrete variable could lead us to very important conclusions like understanding data distribution across different levels of a categorical variable. A bivariate plot between two discrete variables could also be developed.

Scatter Plot

Scatter plots or scatter graphs is a bivariate plot having greater resemblance to line graphs in the way they are built. A line graph uses a line on an X-Y axis to plot a continuous function, while a scatter plot relies on dots to represent individual pieces of data. These plots are very useful to see if two variables are correlated. Scatter plot could be 2 dimensional or 3 dimensional.

Syntax: seaborn.scatterplot(x=None,y=None, hue=None, style=None, size=None, data=None, palette=None, hue_order=None, hue_norm=None, sizes=None, size_order=None, size_norm=None, markers=True, style_order=None, x_bins=None, y_bins=None, units=None, estimator=None, ci=95, n_boot=1000, alpha='auto', x_jitter=None, y_jitter=None, legend='brief', ax=None, **kwargs) Parameters:
x, y: Input data variables that should be numeric.

data: Dataframe where each column is a variable and each row is an observation.

size: Grouping variable that will produce points with different sizes.

style: Grouping variable that will produce points with different markers.

palette: Grouping variable that will produce points with different markers.

markers: Object determining how to draw the markers for different levels.

alpha: Proportional opacity of the points.

Returns: This method returns the Axes object with the plot drawn onto it.

Advantages of a scatter plot

- Displays correlation between variables
- Suitable for large data sets
- Easier to find data clusters
- Better representation of each data point

Histogram

Histograms display counts of data and are hence similar to a bar chart. A histogram plot can also tell us how close a data distribution is to a normal curve. While working out statistical method, it is very important that we have a data which is normally or close to a normal distribution. However, histograms are univariate in nature and bar charts bivariate.

A bar graph charts actual counts against categories e.g. height of the bar indicates the number of items in that category whereas a histogram displays the same categorical variables in bins.

Bins are integral part while building a histogram they control the data points which are within a range. As a widely accepted choice we usually limit bin to a size of 5-20, however this is totally governed by the data points which is present.

Countplot

A countplot is a plot between a categorical and a continuous variable. The continuous variable in this case being the number of times the categorical is present or simply the frequency. In a sense, count plot can be said to be closely linked to a histogram or a bar graph.

Syntax : seaborn.countplot(x=None, y=None, hue=None, data=None, order=None, hue_order=None, orient=None, color=None, palette=None, saturation=0.75, dodge=True, ax=None, **kwargs)

Parameters : This method is accepting the following parameters that are described below:

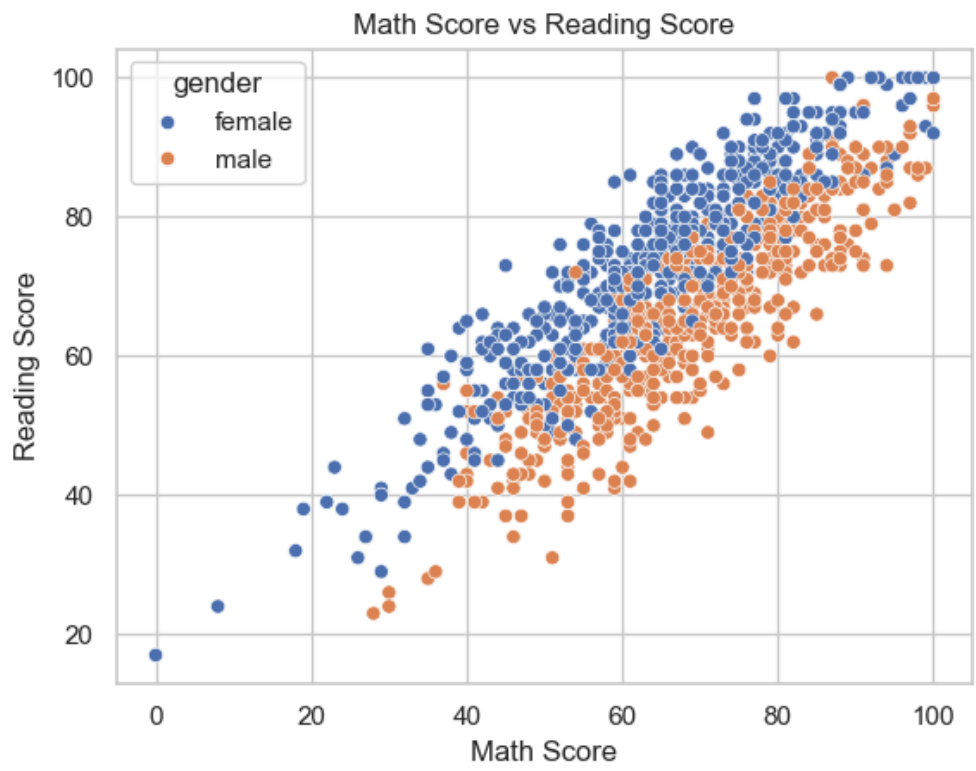
- x, y: This parameter takes names of variables in data or vector data, optional, Inputs for plotting long-form data.
- hue : (optional) This parameter takes column name for colour encoding.
- data : (optional) This parameter take DataFrame, array, or list of arrays, Dataset for plotting.
- order, hue_order : (optional) This parameter take lists of strings. Order to plot the categorical levels in, otherwise the levels are inferred from the data objects.
- orient : (optional)This parameter takes "v" | "h", Orientation of the plot (vertical or horizontal). This is usually inferred from the dtype of the input variables but can be used to specify when the "categorical" variable is a numeric or when plotting wide-form data.
- color : (optional) This parameter takes matplotlib color, Color for all of the elements, or seed for a gradient palette.

- palette : (optional) This parameter takes palette name, list, or dict, Colors to use for the different levels of the hue variable. Should be something that can be interpreted by color_palette(), or a dictionary mapping hue levels to matplotlib colors.
- saturation : (optional) This parameter take float value, Proportion of the original saturation to draw colors at. Large patches often look better with slightly desaturated colors, but set this to 1 if you want the plot colors to perfectly match the input color spec.
- dodge : (optional) This parameter take bool value, when hue nesting is used, whether elements should be shifted along the categorical axis.
- ax : (optional) This parameter take matplotlib Axes, Axes object to draw the plot onto, otherwise uses the current Axes.
- kwargs : This parameter take key, value mappings, Other keyword arguments are passed through to matplotlib.axes.Axes.bar().

Feature	Matplotlib	Seaborn
Functionality	Low-level, highly customizable, requires more code	High-level, built on top of Matplotlib, includes statistical visualization functions
Syntax	More complex, requires multiple lines of code for customization	Simpler, concise, and easier for statistical plotting
Dealing with Multiple Figures	Uses plt.figure() and plt.subplot() for multiple plots	Built-in support for handling multiple plots easily
Visualization	Basic visualizations like scatter, bar, and line plots	Advanced statistical plots like violin plots, pair plots, and heatmaps
Pliability (Flexibility)	Highly flexible with full control over customization	Less flexible but provides automatic enhancements
Data Frames and Arrays	Works with NumPy arrays and Pandas DataFrames but requires manual handling	Natively supports Pandas DataFrames, making it easier to work with structured data
Use Cases	General-purpose plotting, detailed customizations, scientific graphs	Statistical data visualization, quick insights, and exploratory data analysis

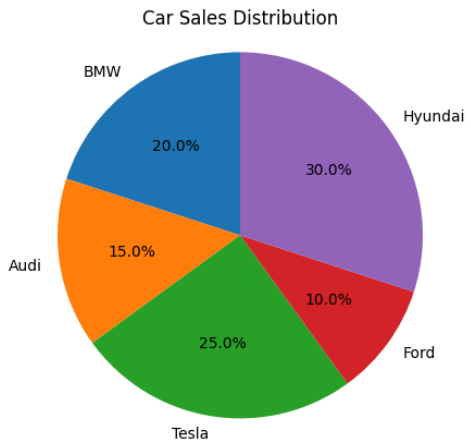
1. Program 1- Scatter plot (StudentsPerformance Data)

```
# Import libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset from CSV file
df = pd.read_csv("DAV\StudentsPerformance.csv")
print(df.head())
sns.set(style="whitegrid")

# 1. Scatter Plot (Math vs Reading)
sns.scatterplot(x='math score', y='reading score', hue='gender', data=df)

plt.title("Math Score vs Reading Score")
plt.xlabel("Math Score")
plt.ylabel("Reading Score")
plt.show()

# 2. Histogram (Math Scores)
sns.histplot(df['math score'], bins=5, kde=True)
plt.title("Distribution of Math Scores")
plt.xlabel("Math Score")
plt.ylabel("Frequency")
plt.show()

# 3. Count Plot (Gender)
sns.countplot(x='gender', data=df)
plt.title("Count of Students by Gender")
plt.xlabel("Gender")
plt.ylabel("Count")
plt.show()
```



2. Program - pie chart to show cars data using Matplotlib

```
# Import libraries
import pandas as pd
import matplotlib.pyplot as plt

# Load dataset from CSV
df = pd.read_csv("DAV/cars.csv")
print(df)

# Create pie chart
plt.pie(df['Sales'], labels=df['Car'], autopct='%1.1f%%', startangle=90)
plt.title("Car Sales Distribution")

# Equal aspect ratio ensures circle shape
plt.axis('equal')
plt.show()
```



Conclusion-

1) What is the difference between matplotlib and seaborn?

Feature	Matplotlib	Seaborn
Level	Low-level library	High-level library
Ease of Use	Requires more code	Simple and concise
Visualization Style	Basic and plain	Attractive and modern
Built On	Independent	Built on Matplotlib

2) Which library is used to create statistical graphics in Python?
The library used to create statistical graphics in Python is **Seaborn**. Seaborn is specifically designed for **statistical data visualization**
It provides built-in functions for:

- Distribution plots (histogram, KDE)
- Regression plots
- Heatmaps
- Boxplots and violin plots

3) Which function is used to create a histogram in Seaborn?

The function used to create a histogram in **Seaborn** is: sns.histplot()